



Верзионисање и failure-first тестирање

Систем који рачуна на кашњење и дупликате

Тема и проблем

Уградити неизвесност мреже у уговор, тест и образложење понашања.

Поруке могу каснити, стићи поново или ван редоследа. Локално успешан тест не доказује да сарадња компоненти остаје исправна.

НАПОМЕНА

Контролисани failure сценарији откривају границе система док су мале и разумљиве, а верзионисање чува кориснике старог уговора.

Четири појма теме

У дистрибуираном систему нормалан сценарио никада није цела прича.

01

Промена

нови уговор или понашање

02

Компатибилност

шта стари корисници разумеју

03

Failure сценарио

контролисана сметња

04

Тест

доказ очекиваног исхода

Промена уговора

Свака промена поруке утиче на бар једног пошиљаоца или примаоца. Пре објаве треба знати да ли старији корисник и даље може да сарађује.

- 01** Утиче на бар једног пошиљаоца или примаоца.
- 02** Објављена верзија се не мења у месту.
- 03** Старији корисник мора и даље моћи да сарађује.

Компатибилност

Компатибилност се посматра из угла примаоца: може ли да разуме нову или стару поруку без погрешне пословне интерпретације?

- 01 Посматра се из угла примаоца, не пошиљаоца.
- 02 Breaking промена се не сме означити као компатибилна.
- 03 Погрешна интерпретација је гора од одбијања поруке.

Failure сценарио

Failure сценарио намерно уводи кашњење, дупликат, прекид или промењен редослед. Тиме се проверава понашање које мрежа реално може изазвати.

- 01** Намерно уводи кашњење, дупликат или прекид.
- 02** Проверава реално мрежно понашање, не идеалан случај.
- 03** Тестира се пре интеграције, не тек на крају.

Тестирани исход

Тест треба да провери видљив пословни исход, а не само одсуство изузетка. То показује да сметња није променила значење операције.

- 01 Проверава видљив пословни исход, не само одсуство грешке.
- 02 Показује да сметња није променила значење операције.
- 03 Основа је за manual-core-baseline пре AI фазе.

Пример: потврда стиже након истека рока

Компонента чека одговор до истека рока и покреће безбедан опоравак.

- 01 Timeout значи да одговор није стигао на време, не да се радња сигурно није извршила.
- 02 Дупликат и касна порука морају се раздвојити од нове легитимне операције.
- 03 Поновљив симулатор омогућава да се исти редослед грешке провери након сваке измене.

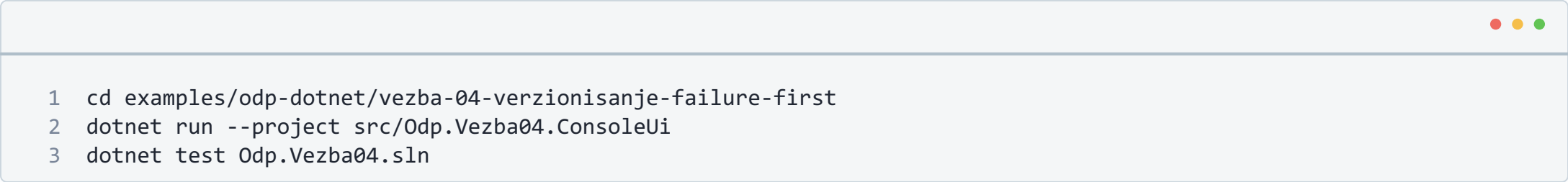
Шема failure-first провере

Циљ није претпоставити савршен транспорт, већ јасно дефинисати понашање када транспорт није савршен.

Елемент	Улога у току
Нормалан ток	захтев и очекиван одговор
Контролисана сметња	кашњење, дупликат или прекид
Видљив исход	стање које систем задржава
Тест	доказ поновљивог понашања

Покретање примера

Пример полази од онога што мрежа сигурно ради: иста порука може стићи два пута, може закаснити и може бити послата новијом верзијом уговора.



```
1 cd examples/odp-dotnet/vezba-04-verzionisanje-failure-first
2 dotnet run --project src/Odp.Vezba04.ConsoleUi
3 dotnet test Odp.Vezba04.sln
```

Правило у коду

Дупликат, закаснала порука и друга мајор верзија су очекивани улази са сопственим кодом.

```
1 public InboxDecision Decide(MessageEnvelope envelope, bool alreadyProcessed, DateTimeOffset now)
2 {
3     ArgumentNullException.ThrowIfNull(envelope);
4
5     if (envelope.Version.Major != rules.SupportedMajor)
6         return new(false, InboxCodes.UnsupportedMajorVersion);
7     if (alreadyProcessed)
8         return new(false, InboxCodes.Duplicate);
9     if (now - envelope.SentAt > rules.MaxAge)
10        return new(false, InboxCodes.TooOld);
11
12    return new(true, InboxCodes.Accepted);
13 }
```

Тест који задаје квар

Пословни ефекат се дешава једном, колико год пута да иста порука стигне.

```
1 [Test]
2 public void Receive_WhenTheSameMessageIsDeliveredTwice_ProcessesItOnce()
3 {
4     var envelope = Envelope("m-1", 1, Start);
5
6     var first = _handler.Receive(envelope);
7     var second = _handler.Receive(envelope);
8
9     using (Assert.EnterMultipleScope())
10    {
11        Assert.That(first.Code, Is.EqualTo(InboxCodes.Accepted));
12        Assert.That(second.Code, Is.EqualTo(InboxCodes.Duplicate));
13        Assert.That(_processor.ProcessedPayloads, Has.Count.EqualTo(1));
14    }
15 }
```

Испис демонстрације

Новија minor верзија се прихвата; друга major верзија се одбија.

```
1 m-1 v1.0 (prva isporuka) -> message_accepted
2 m-1 v1.0 (ponovljena isporuka iste poruke) -> message_duplicate
3 m-2 v1.3 (novija minor verzija) -> message_accepted
4 m-3 v2.0 (druga major verzija) -> message_major_version_unsupported
5 m-4 v1.0 (poruka koja je putovala 12 minuta) -> message_too_old
6
7 Poslovni efekat se dogodio 2 puta:
8   telemetrija m-1
9   telemetrija m-2
```

Принципи и замке

Принцип важи тек када је препозната и замка која га тихо поништава.

ПРИНЦИПИ

Компатибилност се процењује према примаоцима поруке, не само према коду пошиљаоца.

Кашњење и дупликат су нормални улази које компонента мора да објасни.

Тест треба да провери видљив исход, не само да метода не баца изузетак.

ЗАМКЕ

Претпоставка да се порука испоручује тачно једном.

Breaking промена означена као безопасна јер локални тестови пролазе.

Тестирање прекида тек након интеграције свих компоненти.

Типичне грешке

Ове грешке се најчешће понављају када се принцип површно примени.

- 01** Претпоставка о испоруци тачно једном.
- 02** Breaking промена означена као компатибилна.
- 03** Прекид се тестира тек на крају интеграције.

Како се принцип примењује

Кораци показују како се принцип препознаје и доследно спроводи у развоју.

- 01** За важан ток навести кашњење, дупликат и прекид.
- 02** Покретати исте експерименте више пута.
- 03** Проверити исход уместо одсуства изузетка.

Питање за дискусију

Одговор се образлаже failure сценаријем и тестом, а не описом срећног пута.

За коју радњу би касно пристигла порука била опаснија од изгубљене и како би је систем препознао?

ЗА ВЕЖБУ

За коју операцију би касно пристигла порука била опаснија од потпуно изгубљене поруке и зашто?

Задатак уз пример

Проширење примера уз вежбу; правило иде у домен, а доказ у тест.

- 01 Додати правило да се порука са `minor` верзијом већом од познате прихвата, али се исход означава новим кодом `message_accepted_newer_minor`, како би тим знао да уговор треба прегледати.
- 02 Покрити гранични случај тестом.

Контролна тачка P2 (10.03.)

Друга контролна тачка (Вежба 3–4) проверава да су идентитет, audit, observability и конфигурација стабилни пре оперативних токова.

- 01 Провера дозволе захтева и улогу и припадност мисији.
- 02 Audit евиденција је append-only и претражива по correlation id-y.
- 03 Message contract registry спречава измену објављене верзије у месту.
- 04 Failure симулатор омогућава поновљиво увођење кашњења и дупликације.

Резиме

За најважније токове тим формулише неколико малих failure експеримената и покреће их у истом окружењу.

Циљ није претпоставити савршен транспорт, већ јасно дефинисати понашање када транспорт није савршен.

НАПОМЕНА

Наредна вежба: Ток података и read модели.